

CCPL CYBER SECURITY INTERNSHIP

Local,

LLM-Driven

SAST/DAST Testing Tool

Initial Design and Technology Decisions

Prepared For	CCPL Internship Supervisor
Prepared By	Muhammad Saqib Taha Asif Minhas
Environment	Local virtual machine with 16 GB RAM
Inference	CPU-based local inference using Ollama
Date	30 July 2026

Current Design Direction

Core scope: Four fixed pipelines - Web SAST, Web DAST, Mobile SAST and Mobile DAST.

Selected stack: HTML/CSS/JavaScript, FastAPI, Ollama and Qwen3 8B Q4_K_M.

Resource rule: One resource-intensive scan at a time on the 16 GB VM.

Report rule: Only reviewed findings appear in the main report; rejected items remain auditable.

1. Introduction.....	3
2. Problem Statement.....	3
3. Functional Requirements.....	3
3.1 Supported Pipelines.....	3
4. Non-Functional Requirements.....	4
5. High-Level Architecture.....	4
6. Technology Selection.....	5
6.1 Frontend.....	5
6.2 Backend.....	5
6.3 Local LLM Runtime.....	6
6.4 Model Selection.....	6
6.5 Quantization Strategy.....	7
7. Scanner Selection.....	7
7.1 Target Handling.....	7
8. AI Pipeline.....	7
.....	9
8.1 Standard Finding Fields.....	10
8.2 Failure Handling.....	10
9. Design Snapshot.....	10
10. Conclusion.....	11

1. Introduction

Project aim: build a locally hosted proof-of-concept that runs SAST or DAST against a fixed web or Android target and produces a reviewed security report.

Supported targets: DVWA for web testing and DIVA for Android testing. All scanning, model inference and report processing will remain local.

Team approach: we will first complete one end-to-end pipeline, then reuse the same normalization, LLM review and reporting components for the remaining three pipelines.

2. Problem Statement

Security tools provide useful evidence, but their outputs are different and may include duplicates, weak alerts or false positives. Our project must turn those raw results into one clear and consistent report.

- **Route** the selected target and test type to the correct local tool.
- **Convert** tool-specific output into one standard JSON structure.
- **Use** a local LLM to explain and assess each candidate finding.
- **Apply** a second review before a finding is accepted as confirmed.
- **Work** within a 16 GB VM without running several heavy scans at once.

3. Functional Requirements

Requirement	Planned Behavior	Output
Guided selection	Show target type, test type and fixed target in sequence.	One valid pipeline choice.
Scan creation	Validate the choice, create scan_id and start a background job.	Immediate acknowledgement.
Status display	Show queued, scanning, LLM review, reporting, completed or failed.	Clear progress on the UI.
Local assessment	Send normalized evidence to the local Assessor prompt.	Initial structured finding.
False-positive review	Review the original evidence and Pass 1 result independently.	Confirmed, rejected or manual review.
Final reporting	Render confirmed findings with severity, location, evidence and remediation.	HTML/JSON report plus optional appendix.

3.1 Supported Pipelines

Pipeline	Primary Input	LLM Review Focus
Web SAST	DVWA source code and configuration	Code patterns, data flow and missing controls.
Web DAST	ZAP alerts plus HTTP request/response evidence	Evidence of exploitable runtime behavior.

Pipeline	Primary Input	LLM Review Focus
Mobile SAST	DIVA APK decompiled code, manifest and resources	Hardcoded secrets, insecure storage, exported components and weak crypto.
Mobile DAST	MobSF/emulator runtime, traffic and logs	Runtime behavior and mobile attack surface.

4. Non-Functional Requirements

Requirement	Design Decision
Locality and privacy	Scanners, model weights, Ollama and reports stay inside the local environment.
Usability	The UI remains guided and simple; the user always sees the current scan stage.
Resource efficiency	Only one heavy scan job runs at a time on the 16 GB VM.
Responsiveness	The scan runs as a background job while FastAPI answers status and result requests.
Reliability	Timeouts, missing tools and model errors produce a clear failed state.
Auditability	Raw, normalized, confirmed and rejected findings are saved locally.
Maintainability	Scanners, parsers, prompts and reporting are separate modules.
Controlled scope	The first version only accepts DVWA and DIVA; arbitrary targets are excluded.

5. High-Level Architecture

Architecture style: a local orchestration pipeline. The frontend sends the selection to FastAPI; the backend runs one scanner, normalizes the output, calls two logical LLM passes and builds the report.

Important execution decision: the security stages remain sequential because each stage needs the previous result. Background execution is used only to keep the API and status screen responsive.

Initial storage: job status is kept in memory, while raw results and reports are saved as local files. A database is not needed for the first proof-of-concept.

Endpoint	Purpose	Response
POST /api/scan	Create and start a scan job.	202 + scan_id
GET /api/scan-status/{scan_id}	Return the current stage or error.	Status JSON
GET /api/scan-results/{scan_id}	Return the completed report.	Report metadata/data

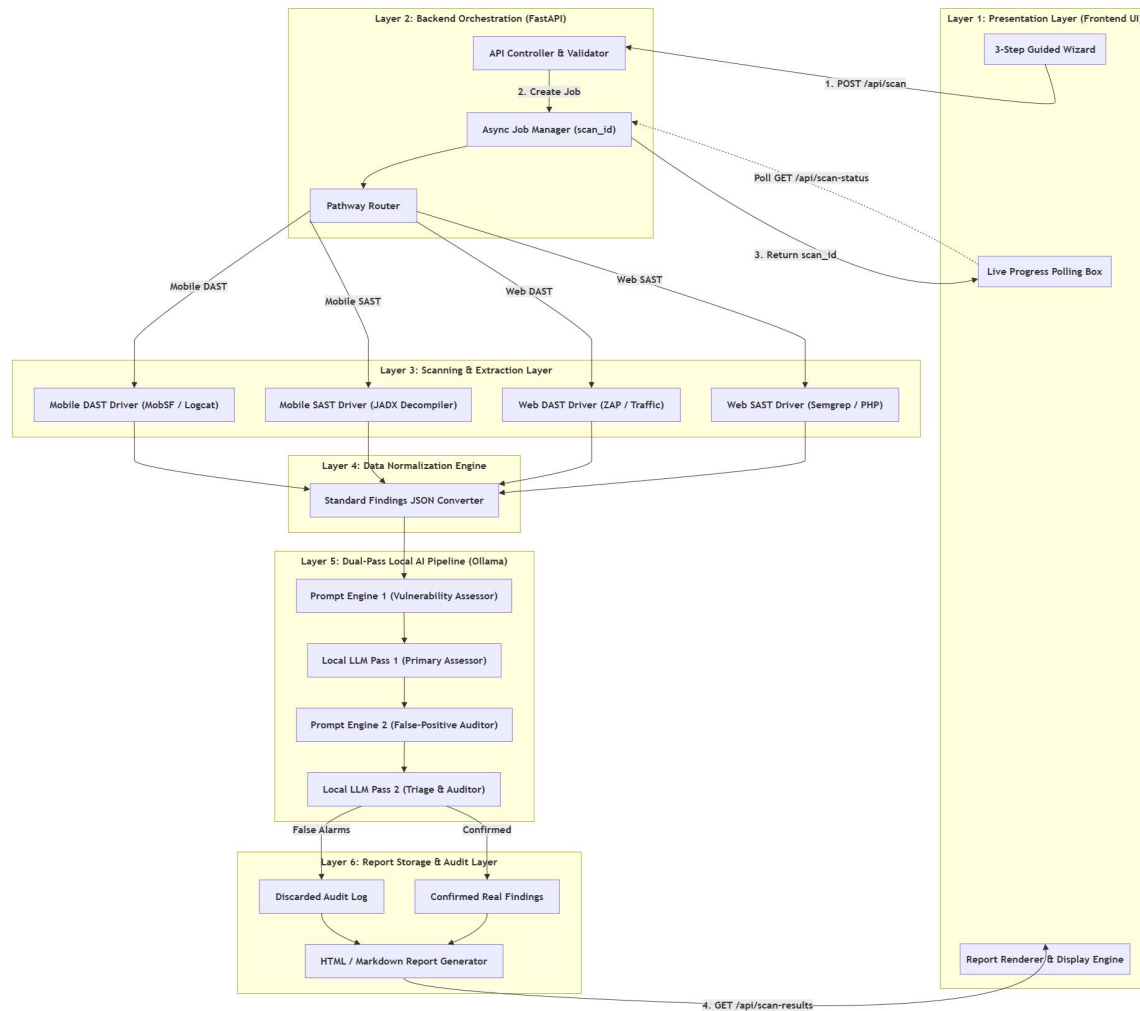


Image 1 Figure 1. High-Level Layered System Architecture of the Proposed AI-Assisted SAST/DAST Vulnerability Assessment Tool

Figure 1 presents the high-level layered architecture of the proposed AI-assisted SAST/DAST Vulnerability Assessment Tool. The architecture is organized into six logical layers, beginning with the Presentation Layer, where users initiate scans and monitor progress, followed by the Backend Orchestration Layer responsible for request handling and scan routing. The selected security scanner performs the analysis, after which the findings are normalized into a standard JSON format. The normalized results are then processed by a dual-stage local AI pipeline for vulnerability assessment and false-positive validation before the Report Generation Layer produces the final security report for presentation to the user.

6. Technology Selection

6.1 Frontend

Selected: HTML5, Vanilla CSS and Vanilla JavaScript.

- Guided choices, status polling and report viewing are simple enough for Vanilla JavaScript.

6.2 Backend

Selected: FastAPI.

- Python was selected because the project is centered around AI integration, API orchestration, automation and security tooling. Most AI frameworks, local inference runtimes and security tools provide first-class Python

support, making integration simpler and reducing development effort. Background jobs keep status and health endpoints responsive.

Feature	FastAPI	Django
Primary Use	REST APIs	Full-stack Web
Performance	High/Fast	Moderate
Async Support	Native	Available but heavier
Swagger Docs	Built-in	Extra packages
AI Integration	Excellent	Good
Project Fit	Excellent	Good

Async Does Not Mean Parallel Scanning

Ordered job: Scanner -> Normalizer -> Assessor -> Reviewer -> Report.

Responsive API: status requests continue, while a single-worker limit protects the 16 GB VM.

6.3 Local LLM Runtime

Selected: Ollama.

Two local inference runtimes were considered: Ollama and LM Studio. Although both can run identical open-source models, Ollama was selected because it provides a REST API, simple CLI management and straightforward backend automation. LM Studio is better suited for interactive experimentation through a graphical interface.

6.4 Model Selection

Selected model: Qwen3 8B.

The model must understand code and scanner evidence, form a security reason and return structured output.

Model	Q4 Working Memory*	CPU Speed	Strength	Project Fit
Gemma 3 4B	~5-7 GB	Very fast	Good for simple tasks	May be weak for complex review.
Qwen2.5-Coder 7B	~6-8 GB	Fast	Excellent code focus	Strong SAST option.
Qwen3 8B - Selected	~7-10 GB	Fast-moderate	Reasoning + code + structured output	Best balance for our 16 GB VM.
Llama 3.1 8B	~7-10 GB	Fast-moderate	Strong general reasoning	Good alternative.
Gemma 3 12B	~10-14 GB	Moderate-slow	Stronger writing/reasoning	Limited memory headroom.
Qwen2.5 14B	~11-15 GB	Slow	Very strong analysis	Too heavy with scanners on this VM.

**Working-memory values are planning estimates. Actual use depends on context length, cache, operating-system overhead and active tools.*

Why Qwen3 8B

Reasoning: strong enough for vulnerability explanation and false-positive review.

Code understanding: suitable for PHP, SQL and Android evidence.

Hardware fit: practical on a 16 GB VM when quantized and used one job at a time.

Output control: well suited to structured prompts and JSON responses.

6.5 Quantization Strategy

Selected: Q4_K_M quantization.

- The model remains an 8B-parameter model; parameters are not removed.
- Weights are stored at lower numerical precision, reducing file size and RAM use.
- The trade-off is a small possible quality loss in exchange for more practical CPU inference.

7. Scanner Selection

Pipeline	Selected Tool	Integration	Role
Web SAST	Semgrep	CLI + JSON output	Scan DVWA source and return candidate code findings.
Web DAST	OWASP ZAP/Burp	Local API/automation	Crawl and test the running DVWA instance.
Mobile SAST	JADX	CLI decompilation	Extract code, manifest and resources from DIVA.apk.
Mobile DAST	MobSF + emulator	Local dynamic analysis	Collect runtime, traffic and log evidence.

7.1 Target Handling

- DVWA SAST uses the local source-code folder.
- DVWA DAST uses the URL of the locally running application.
- DIVA SAST uses the APK file without running the app.
- DIVA DAST requires the APK to run in an emulator or approved test device.

Boundary: the tools collect evidence; the local LLM makes the vulnerability judgment.

8. AI Pipeline

1	Collect Evidence - Run the selected scanner and preserve its raw output.
2	Normalize - Convert tool-specific fields into one Standard Findings JSON format.

3

LLM Pass 1 - Assessor - Explain the issue, review context, estimate severity/confidence and suggest remediation.

4

LLM Pass 2 - Reviewer - Check the original evidence and decide confirmed, rejected or manual review.

5

Report - Place only confirmed findings in the main report and keep rejected items in an appendix/log.

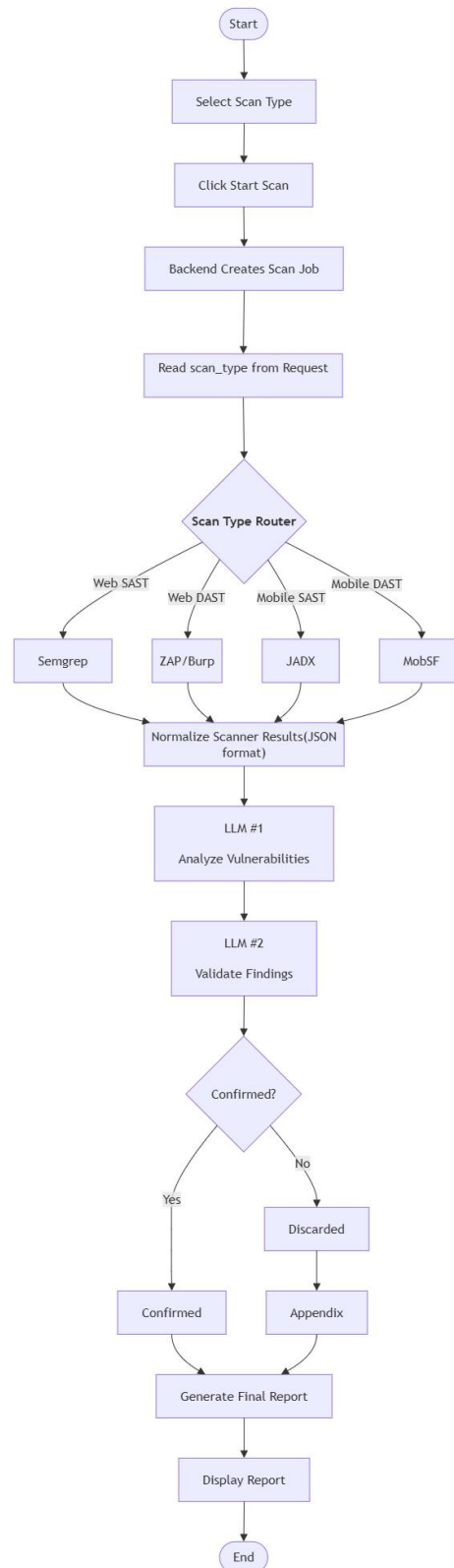


Image 2Image 2Figure 2 — AI-Based Vulnerability Assessment Workflow

8.1 Standard Finding Fields

Identity	Evidence	Assessment	Decision
scan_id, target type, test type, tool	title, location, code/HTTP/runtime context	severity, confidence, reasoning, remediation	confirmed, rejected or manual review

8.2 Failure Handling

Failure	Planned Response
Scanner missing/crashes	Mark failed, save logs and stop before LLM review.
Scanner timeout	Terminate where possible and show a timeout status.
Ollama/model unavailable	Save evidence, mark AI stage failed and do not confirm alerts.
Invalid LLM JSON	Validate, retry once and then send to manual review.
Job already active	Queue or return a busy response; do not start another heavy job.
Browser refresh	Continue the backend job and reconnect using scan_id in localStorage.

9. Design Snapshot

Area	Decision	Reason
Scope	Local proof-of-concept	The priority is the reusable testing pipeline, not arbitrary target input.
Targets	DVWA and DIVA	Both are known vulnerable applications suitable for controlled testing.
Backend	FastAPI	Fits Python-based AI/tool orchestration and status APIs.
LLM	Qwen3 8B Q4_K_M via Ollama	Balanced code understanding, reasoning and memory use.
Execution	Sequential pipeline + responsive status API	Stages depend on each other; the UI must still remain responsive.
Storage	Local JSON/HTML files; no database initially	The first version has no login, scan history dashboard or multi-user data.

Document basis: This design translates the supplied project requirements into a practical implementation plan for our team.

10. Conclusion

Final design direction: a simple local interface, FastAPI orchestration, one selected scanner, standardized evidence, Qwen3 8B Q4_K_M through Ollama, an independent second review and a clear final report.

Why this fits our environment: it keeps the workload controlled on the 16 GB VM and avoids unnecessary components such as a database, concurrent heavy scans or separate model instances.

Next implementation milestone: complete one vertical slice - Web SAST on DVWA using Semgrep -> normalization -> Assessor -> Reviewer -> local report. The other pipelines will then reuse the same core.